Persistence Diagrams as Barycenters for Wassertein Geodesics

Report on *Wasserstein Dictionaries of Persistence Diagrams*

Matthieu Pierre Boyer*
École Normale Supérieure
Paris, France
matthieu.boyer@ens.fr

Gary Klajer*
ÉNS Paris-Saclay
Gif-sur-Yvette, France
gary.klajer@ens-paris-saclay.fr

Marc Perlade*
École Normale Supérieure
Paris, France
marc.perlade@ens.fr

Figure 1

Abstract

In this report based on [SDT23], we study the idea of doing non-linear dimensionality reduction on a large dataset of persistence diagrams, using atomic persistence diagrams that span the dataset for the hull defined by the optimal transport of diagrams. We will first give the mathematical intuition on why this could work then provide an algorithm that proposes a method to compute such a representation. We will finally provide benchmarks on the different methods, implemented in Rust.

Keywords

Persistence, Dictionaries, Optimization, Topological Data Analysis

Introduction

When considering structured datasets mapping to an underlying geometric structure, computing topological invariants as is done in topological data analysis will lead to the study of persistence diagrams, large sets of points of the plane mapping to the stability and impactfullness of topological features when considering more and more of the points in the dataset. This yields an accurate mapping of the topological homology structure[1] generated by a meshing of the dataset and thus by the dataset when the meshing is accurate enough.

Persistence diagrams are thus fundamental tools to understand the geometric structure of data, and have many practical applications, from clustering (ToMaTo algorithm by Chazal et al.) to non-rigid shape segmentation. However, such objects live in a highly non-linear space and might have prohibitive quantities of points to run machine learning algorithms on.

[SDT23] suggests using optimal transport to define atomic diagrams that allows to represent specific persistence diagrams as barycenters for the Wassertein geodesics as a way to do non-linear dimensionality reduction over some set of base diagrams that we want to study.

1 Mathematical preliminaries

In this section we will lay out the different mathematical objects we will use when presenting the algorithm that allows us to optimize our atomic dictionary.

We are given a set $X = \{X_1, \dots, X_n\}$ of N persistence diagrams. Each input diagram is given as the set of its persistence pairs mapped to points in the 2-dimensional plane[2]. In this sense, each diagram X_i is given as a finite set in $\mathbb{R}^2$.

We will also consider a cost function between the points of the diagrams by considering the squared ℓ^2-norm on $\mathbb{R}^2$. This will be the main comparison point between two persistent features.

Optimal transport. Optimal transport is the mathematical field concerned with mapping (dis-)similarities from a space with some sense of geometry to the space of probability distributions on that space. More precisely, given two measures α, β on spaces $\mathcal{X}$ and $\mathcal{Y}$, as well as a cost function $c : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, it aims at computing the optimal transport plan $T_\sharp \alpha = \beta$,

$$M = \inf_{T_\sharp \alpha = \beta} \int c(x, T(x)) \mathrm{d}\alpha(x)$$

where $T_\sharp \alpha(B) = \alpha\left(T^{-1}(B)\right)$ for $T : \mathcal{X} \to \mathcal{Y}$ that maps points to $\mathcal{X}$ to points in $\mathcal{Y}$ minimising the transport cost and such that density of α is correctly taken to density in β.

When $\mathcal{X} = \mathcal{Y}$ and c is some power p of a metric on $\mathcal{X}$, then the optimal transport cost $M(\alpha, \beta)$ becomes a metric W_p on the space $\mathcal{P}(\mathcal{X})$ of probability measures on $\mathcal{X}$. This gives us a notion of geodesics on $\mathcal{P}(\mathcal{X})$ and as such a notion of barycenters as minimisers of the Fréchet energy

$$E_F\left(X, (X_i), (\lambda_i)\right) = \sum_{i=1}^{n} \lambda_i W_p^p(X, X_i),$$

where $(\lambda_1, \dots, \lambda_n) \in \Delta_n$ the probability simplex of size n. In the case studied here, we will take the ℓ^2 norm and thus consider $p = 2$ in the above equation.

One can see a persistence diagram X_i as a uniform probability distribution over its points in $\mathbb{R}^2$. However, in the above definition, one cannot *split mass* present on one point, meaning that if $|X_i| \not\sim |X_j|$, there will not be any transport plan between X_i and X_j. To remove that issue, we will provide a notion of augmentation on diagrams. Notice that for persistence diagrams, points $(b, d) \in \mathbb{R}^2$ where $b = d$ do not have any real topological feature associated. As such, we will augment the set of points in X_i by adding to it the projections on the diagonal of $\mathbb{R}^2$ of each point in each of the $(X_j)_{j \neq i}$.

This will make all diagrams have the same number of points. We then simply need to extend our cost function to be zero between two points in the diagonal (again, this comes from the fact that a point on the diagonal can be seen, as any other point on the diagonal, as

*All authors contributed equally.

[1] We will always consider homology on $\mathbb{F}_2$ for simplicity.

[2] Or half-plane above the line $y = x$.

a *fake* topological feature) and we can compute transport plans in this setting, with a cost function that is convex.

Matchings for 2-dimensional transport plans. Computing efficiently the transport plan between two diagrams will be crucial when evaluating barycenters. Indeed, as we will see in the next section, we aim run a gradient descent on atoms and weights computing barycenters of the set of base diagrams. To do so, we simply consider the problem of computing an optimal assignation from diagram X_i to diagram X_j as a matching problem. Indeed, since $|X_i| = |X_j|$ after augmentation, a transport plan from the uniform distribution on X_i to the one on X_j is purely an assignation on the points of X_i to the points of X_j that minimizes the cost.

This problem is formally modeled by considering the complete bipartite graph $X_i \sqcup X_j$ with the cost matrix associated to the distance defined earlier. It is then known that this algorithm can be solved optimally in polynomial time by the hungarian algorithm.

2 Optimization of atoms

In this section we will describe the procedure used to compute the atoms $\mathcal{D} = \{a_1, \dots, a_m\}$ and weights $\Lambda = \{\lambda_1, \dots, \lambda_N\}$ for the Wasserstein barycenter description of our dataset $X_1, \dots, X_N$.

2.1 Mathematical principle

The algorithm works as a joint optimization on two sub-problems: the optimization on the atoms $\mathcal{D}$ and the optimization on the vector of barycentric weights for X_i denoted $\lambda_i \in \Delta_m$ in order to minimize the sum of the Fréchet energies (the *dictionary energy*)

$$E_D(\Lambda, \mathcal{D}) = \sum_{n=1}^{N} W_2^2(Y(\lambda_n, \mathcal{D}), X_n),$$

where $Y(\lambda_n, \mathcal{D}) = \underset{Y}{\operatorname{argmin}} E_F(Y, \mathcal{D}, \lambda_n)$.

It will thus work in two gradient descent steps:

(1) The weight optimization for a fixed $\mathcal{D}$;
(2) The dictionary optimization for a fixed Λ.

We refer the reader to Section 3 of [SDT23] for the detail of the computations, as those are not the focus of this report.

Weight optimization. Since the dictionary energy for the weights is separated, we consider here only a single X_i to give the expression of the gradient and the descent step. Since our optimal transport plans are optimal assignments $\phi_1, \dots, \phi_m$ between $Y(\lambda_i, \mathcal{D})$ and the k-th atom a_k, we can write the energy as

$$E_W(\lambda_i) = W_2^2(Y(\lambda_i, \mathcal{D}), X_i) = \sum_{j=1}^{|Y(\lambda_i,\mathcal{D})|} \left\| \sum_{k=1}^{m} \lambda_{i,k} \left(a_k^{\phi_k(j)} - x^{\psi(j)} \right) \right\|^2$$

where ψ is the optimal assignment between X_i and its approximation $Y(\lambda_i, \mathcal{D})$, and $d^{\phi(j)}$ is the $\phi(j)$-th not-diagonal point of diagram d. Finally, we find the gradient of the energy

$$\nabla E_W(\lambda_i) = 2 \sum_{k=1}^{m} \sum_{j=1}^{|Y(\lambda_i,\mathcal{D})|} \begin{bmatrix} \left(a_1^{\phi_1(j)} - x^{\psi(j)} \right)^T \\ \vdots \\ \left(a_m^{\phi_m(j)} - x^{\psi(j)} \right)^T \end{bmatrix} \left(\lambda_{i,k} \left(a_k^{\phi_k(j)} - x^{\psi(j)} \right) \right) \tag{E_W}$$

and we compute the barycentric weights at the next iteration by one step of gradient descent of step ρ_{λ_i} which converges when ρ_{λ_i} is less than one over the lipschitz constant of ∇E_W.

Atom optimization. The dictionary energy being not separated on the atoms but on the dataset, we again consider only one input diagram and repeat the process. Again we have the energy associated to X_i being

$$E_A(\mathcal{D}) = W_2^2(Y(\lambda_i, \mathcal{D}), X_i) = \sum_{j=1}^{|Y(\lambda_i,\mathcal{D})|} \left\| \sum_{k=1}^{m} \lambda_{i,k} \left(a_k^{\phi_k(j)} - x^{\psi(j)} \right) \right\|^2$$

and we can thus compute the pointwise atom energy associated to the atom points $\mathcal{D}^j$ matching to a given point y^j of the barycentric approximation $Y(\lambda_i, \mathcal{D})$ of X_i by

$$e_A(\mathcal{D}^j) = \left\| \sum_{k=1}^{m} \lambda_{i,k} (a_k^{\phi_k(j)} - x^{\psi(j)}) \right\|^2 : weights$$

and thus computing it's gradient

$$\nabla e_A(\mathcal{D}^J) = 2 \begin{bmatrix} \lambda_{j,1} & \cdots & \lambda_{j,m} \end{bmatrix}^T \left(\sum_{k=1}^{m} \lambda_{j,k} \left(a_k^{\phi_k(j)} - x^{\psi(j)} \right)^T \right). \tag{E_A}$$

Finally, we simply iterate a gradient descent with a step $\rho_{\mathcal{D}^j}$ which converges when $\rho_{\mathcal{D}^j}$ is less than one over the lipschitz constant of ∇e_A.

2.2 Implementation and optimizations

The algorithm above was implemented in Rust to try and reproduce the results of the paper. We chose to represent the diagrams as dynamic arrays of points, and implemented as many optimizations as we could.

Barycenter algorithm. In [VBT19], on which [SDT23] is partially based, the authors present a few optimizations for what they call a *progressive* approximation of barycenter for the Wasserstein metric of persistence diagrams.

Algorithm 1 Barycentric Approximation Algorithm

```
function BARYCENTER(D = (a_i, ..., a_m), λ)
    Output: Barycenter Y(λ, D)
    Y(λ, D) ← a_i                          ▷ i randomly chosen in [1, m]
    while E_F decreases and t ≤ t_max do   ▷ Do at most t_max iterations
        for i ∈ [1, m] do
            φ_i ← Auction_δ((a_i)_ρ, Y(λ, D))
        Y(λ, D) ← Update(φ_1, ..., φ_m)    ▷ Arithmetic mean
        δ ← δ/5
        ρ ← PS(ρ)
    return Y(λ, D)
```

They introduce a few main optimizations, not all of which we implemented.

- First, they suggest that the different matching computations retain the previous assignations in memory when creating a matching (see below).

- Moreover, when computing the assignments, progressively increasing the accuracy of the algorithm we use while retaining the previous prices for our initialisation yields better and better result with a good performance for the computation time.
- They suggest that inside actual persistence diagrams present many artifacts introduced by the filtration used to generate them. Those artifacts come in the form of persistence pairs with small difference between their birth and their death. By beginning the optimization process on a diagram where those artifacts are removed, then progressively adding them back by lowering the threshold of "too low persistence", they are able to locally limit the number of close initialized points and thus distribute among the inputs what the new barycenter points should be. To do so, they define for $\rho > 0$ a threshold, the filtered diagram $a_{|\rho}$ consisting of the points $a \in a_i$ such that $y_a - x_a > \rho$. They chose to reduce ρ so that the size of $a_{|\rho}$ does not increase by more than 10% at each iteration.
- Finally, the authors have added parallelism to their algorithm, which comes at the matching step, which clearly is dependent only on the atom considered and not the others. We did not implement parallelism due to time constraints.

Matching algorithm. The main point of optimization is inside the matching computation. Indeed, after each optimization step we need to recompute every barycentric approximation of our points. As such, the classical Kuhn-Munkres (or Hungarian) algorithm will not be sufficient.

Following [KMN16], we implemented a variant of the auction algorithm which makes use of k-d trees, a space partioning data-structure which allows for fast closest-neighbour queries and returns a $1+\delta$ approximation of the result in a bit faster time. This is used when finding the best objects j and k based on their value.

The value $v_{i,j}$ of a point j to bidder i is $-w^2(i,j) - p_j$ where $w^2(i,j)$ is the distance function on the points defined in the previous section and p_j is representing an additional price during the auction process, mimicking that a point j must be *valuable enough* for i to get through the trouble of reassigning j to i instead of its previous assignation.

Algorithm 2 Auction Algorithm for Matching Approximation

function AUCTION(Diagrams X, Y, threshold $\delta > 0$)
  Output: δ-approximate 2-Wasserstein distance $W_2(X,Y)$
  $d \leftarrow 0, \varepsilon \leftarrow \frac{5}{4}$ ▹ Maximum edge length
  while $d^2 > (1+\delta)^2(d^2 - n\varepsilon)$ **do**
    $\varepsilon \leftarrow \frac{\varepsilon}{5}$
    $M \leftarrow \emptyset$ ▹ Empty matching
    while $\exists i, M(i) = \emptyset$ **do**
      $j \leftarrow \operatorname{argmax} v_{i,j}$
      $k \leftarrow \operatorname{argmax}_{k \neq j} v_{i,k}$
      $M^{-1}(j) \leftarrow M^{-1}(j) \setminus \{j\}$
      $M(i) \leftarrow \{j\}$
      $p_j \leftarrow p_j + (v_{i,j} - v_{i,k}) + \varepsilon$
    $d \leftarrow \sqrt{\operatorname{cost}(M)}$
  return d

This comes at the cost of a large overhead computation, but this can be reduced a bit by prize memorization. In this case, this amounts to initializing the prices for the bidders to the ones they had at the last round of auction.

Gradient descent algorithm. The gradient descent algorithm asks that we compute the barycentric approximations for each base diagram in the data set after each substep of the descent. The initialisation of the dictionary is done by taking some sense of *extremal points* for the energy we try to minimize. The barycentric are also defined to be uniform at $\frac{1}{m}$.

Algorithm 3 Wasserstein Dictionary Optimization

function COMPUTEDICTIONARY($X = \{X_1, \dots, X_N\}, m$)
  Output: Dictionary $\mathcal{D} = \{a_1, \dots, a_m\}$, barycentric weights $\Lambda = \{\lambda_1, \dots, \lambda_N\}$
  $\mathcal{D} \leftarrow k\text{-}\underset{a \in X}{\operatorname{argmax}} \sum_{f \in X} W_2(a, f)$
  for $1 \le i \le N$ **do**
    $\lambda_i \leftarrow \begin{pmatrix} \frac{1}{m} & \cdots & \frac{1}{m} \end{pmatrix}$
  for $\tau \in \{\tau_1, \dots, \tau_r\}$ **do**
    while $E_D(\Lambda, \mathcal{D})$ decreases **do**
      for $1 \le n \le N$ **do**
        Gradient step ρ_{λ_n} along ∇E_W at X_n ▹ (E_W)
      for $1 \le n \le N$ **do**
        Gradient step $\rho_{\mathcal{D}}$ along ∇E_A at X_n ▹ (E_A)

In [SDT23] propose a strategy that improves the resolution by removing and progressively re-adding to the input diagrams the artifacts discussed in the progressive barycenter algorithm based on [VBT19]. Their process works by suggesting values for $\tau_1, \dots, \tau_r$ such that

However, computing the matchings between atoms and computing the barycenters of the atoms is easily done in parallel. Indeed, once the atomic dictionary $\mathcal{D}$ is known, the problem is purely dependent on the weights and thus it can be solved independently for each X_i. Then, the computation of the gradients can also be done in parallel, independently for each X_i.

We did not have time to add parallelism in our implementation, but as you will see this did not have such a major impact on the resulting computation time.

3 Analysis and comparison of implementations

References

[KMN16] Michael Kerber, Dmitriy Morozov, and Arnur Nigmetov. Geometry helps to compare persistence diagrams, 2016.
[SDT23] Keanu Sisouk, Julie Delon, and Julien Tierny. Wasserstein dictionaries of persistence diagrams, 2023.
[VBT19] Jules Vidal, Joseph Budin, and Julien Tierny. Progressive wasserstein barycenters of persistence diagrams. *IEEE Transactions on Visualization and Computer Graphics*, page 1–1, 2019.